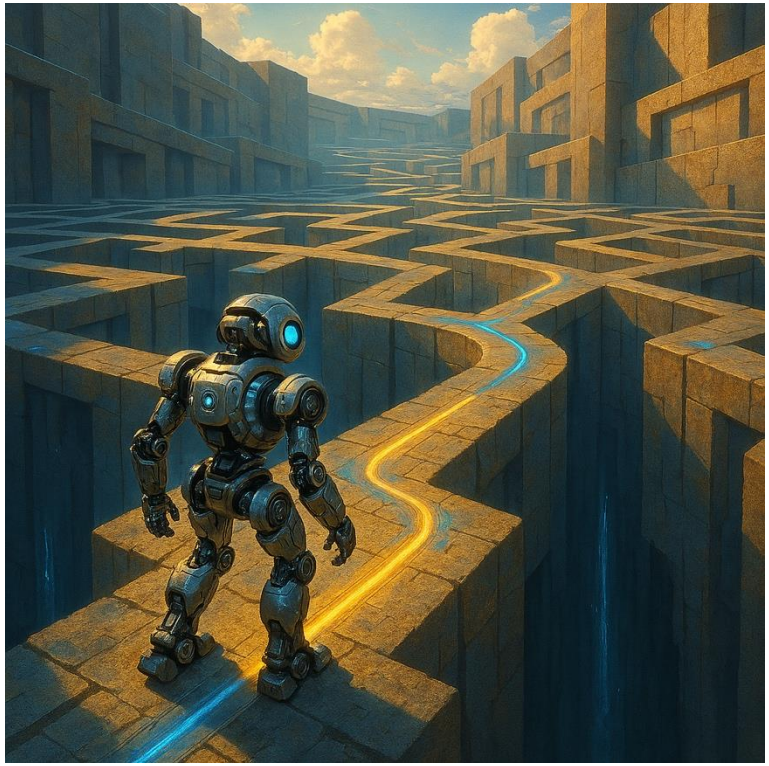
Aprendizado por Reforço

Prof. Dr. Hendrik Macedo
Departamento de Computação

Onde o agente aprende com a experiência



Limitações do Minimax

- Dependência Humana
  - No Minimax clássico, nós (engenheiros) definimos a função $Eval(s)$ e os pesos w_i
  - "Torres valem 5 pontos". Quem disse? E se valerem 5.2?
- Rigidez
  - Se o jogo muda, o agente quebra. Ele não *aprende*, apenas *executa* nossa lógica.
- Como aprender a jogar sem que um humano explique as táticas?
  - Como fazer o agente descobrir sozinho que $w_{torre} = 5$?

Aprendizado por reforço

- O Aprendizado por Reforço (Reinforcement Learning) é um paradigma de aprendizado de máquina onde um agente aprende a tomar decisões através da interação com um ambiente: **aprender por meio da experiência**
- O agente recebe **recompensas** por ações que o levem a atingir seus objetivos e **penalidades** por ações que o afastem deles.
- Através da **tentativa e erro**, o agente aprende a maximizar as recompensas e minimizar as penalidades, desenvolvendo uma **política ótima para navegar no ambiente**.

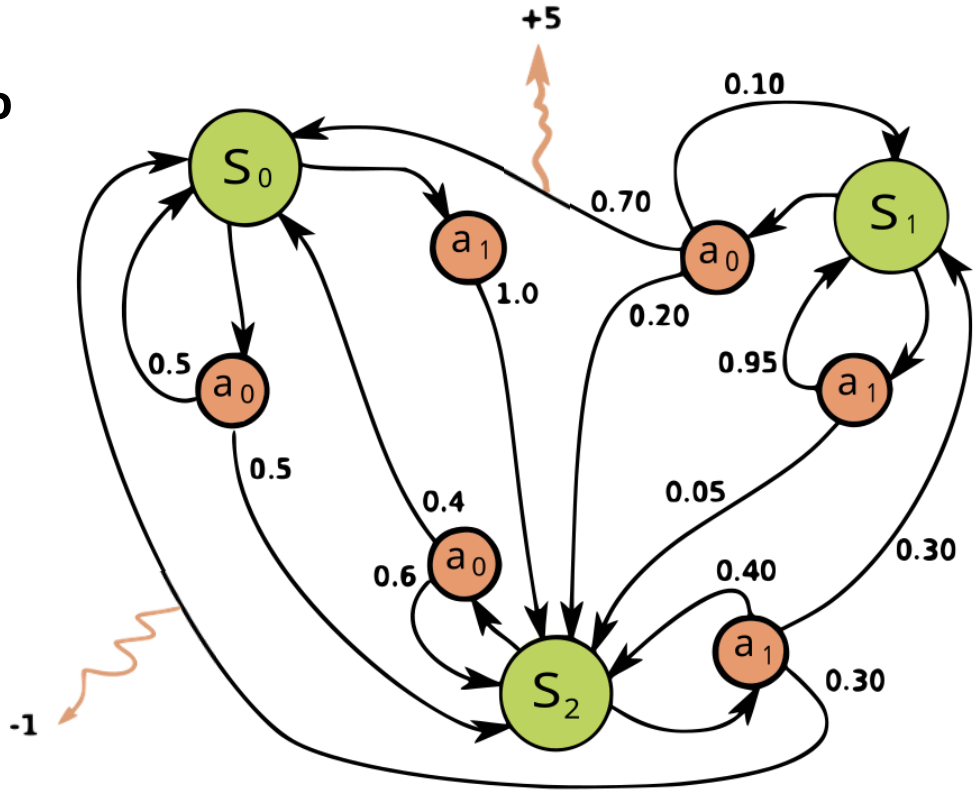
Exemplo: aprender a jogar um novo jogo

- O agente interage com o jogo (ambiente) tomando ações (movimentar o personagem, usar armas, etc.).
- O jogador recebe recompensas (pontos, passar de fase) por ações bem-sucedidas e penalidades (perder vida, game over) por ações malsucedidas.
- **Com o tempo, o jogador aprende quais ações levam a melhores resultados e desenvolve uma estratégia (política) para jogar o jogo com sucesso**.
- Esse processo de aprendizado se assemelha exatamente à maneira como os seres humanos aprendem a jogar por meio de tentativa e erro

Processo de Decisão de Markov (MDP)

- Modelo matemático que descreve um **processo de decisão sequencial**, onde age-se em um espaço de estados, para maximizar uma recompensa acumulada ao longo do tempo, obedecendo à propriedade de Markov, ou seja, a decisão depende apenas do estado atual.

$$(S, A, P_a, R_a)$$



$P(s'|s,a)$ **: probabilidade de ir para o estado** s' **a partir do estado** s **tomando a ação** a**.**

$R(s'|s,a)$ **: recompensa recebida ao ir para o estado** s' **a partir do estado** s **tomando a ação** a**.**

Formalização do MDP

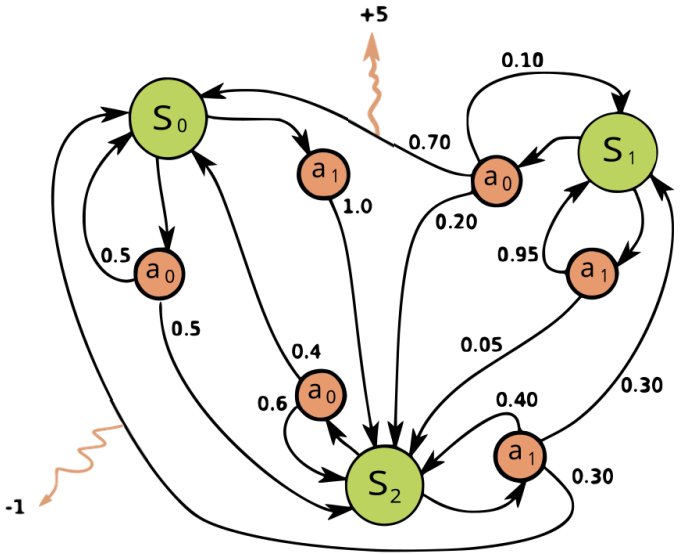
- O objetivo do agente em um MDP é encontrar uma **política ótima π** que maximize a recompensa acumulada ao longo do tempo.
- A política define qual ação o agente deve tomar em cada estado.

$$a_t = \pi(s_t)\ \forall s$$

Valor esperado de recompensa

$$a_t = \pi(s_t)\ \forall s$$

$$E\left[\sum_{t=0}^{\infty} R_{a_t}(s_t, s_{t+1})\right]$$



Fator de desconto $\mathbf{0 \leq \gamma \leq 1}$: importância das recompensas futuras em relação às recompensas imediatas.

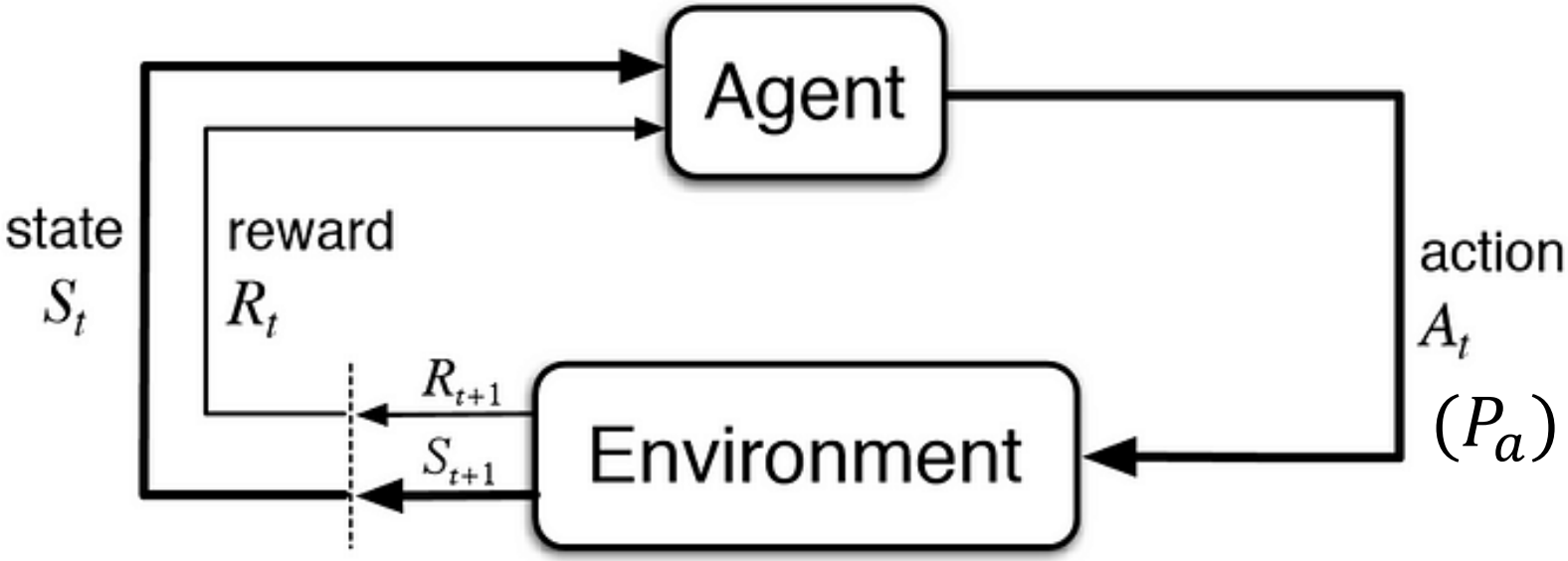
$$E = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Se $\gamma \approx 0$?

Se $\gamma \approx 1$?

Mas aprender o quê?

MDP: (S, A, P_a, R_a)



$$(S, A, P_a, R_a)$$

? ?

Tipos de Aprendizado por Reforço

- **Categorias** de algoritmos
  - **Baseado em modelo:** O agente possui um modelo do ambiente, que permite prever as consequências de suas ações.
  - **Livre de modelo:** O agente não possui um modelo do ambiente e aprende diretamente da experiência.
- **Políticas** dos algoritmos
  - **On-policy:** O agente aprende a partir da política que está sendo utilizada no momento.
  - **Off-policy:** O agente aprende a partir de uma política diferente da que está sendo utilizada no momento.

Algoritmo Q-learning

- Algoritmo **livre de modelo** e **off-policy**
- O agente aprende a atribuir valores a cada ação (a) que pode tomar em um determinado estado (s).
- A Função $Q(s, a)$ representa a **qualidade** (valor ou utilidade) de tomar uma **ação ($\boldsymbol{a}$)** em um **estado ($\boldsymbol{s}$)** específico, em termos da **recompensa futura esperada e descontada**.
- Q é o coração do algoritmo Q-Learning.
- Os valores Q são armazenados em uma **tabela Q, que é atualizada iterativamente** à medida que o agente interage com o ambiente.
- O agente explora o ambiente tomando ações e observando as recompensas e os estados resultantes.
- A cada passo, o agente atualiza o valor Q do par estado-ação anterior usando a **equação de Bellman**

Equação de Bellman

$$Q(s,a) \leftarrow (1-\alpha)\underbrace{Q(s,a)}_{\text{Experiência anterior}} + \alpha\underbrace{[R(s,a,s') + \gamma \max_{a'} Q(s',a')]}_{\text{Experiência atual}}$$

onde:

$Q(s,a)$: o valor Q do par estado-ação

: a recompensa recebida por tomar a ação a no estado s

γ : o fator de desconto

$\max_{a'} Q(s',a')$: o valor Q máximo para o próximo estado s'

α : **Taxa de Aprendizado,** que define o quanto aproveitar da experiência atual

$$0 \leq \alpha \leq 1$$

Algoritmo Q-learning

1. Inicialize $Q(s, a)$ arbitrariamente para todos os estados s e ações a
2. Para cada episódio:
   3. Selecione um estado s aleatoriamente
   4. Enquanto s não for um estado terminal:
      5. Escolha a ação a partir de s usando um critério qualquer
      6. Execute a ação a e observe a recompensa R e o próximo estado s' (amostra)
      7. Atualize $Q(s, a)$ usando a equação de Bellman:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} Q(s', a')]$$

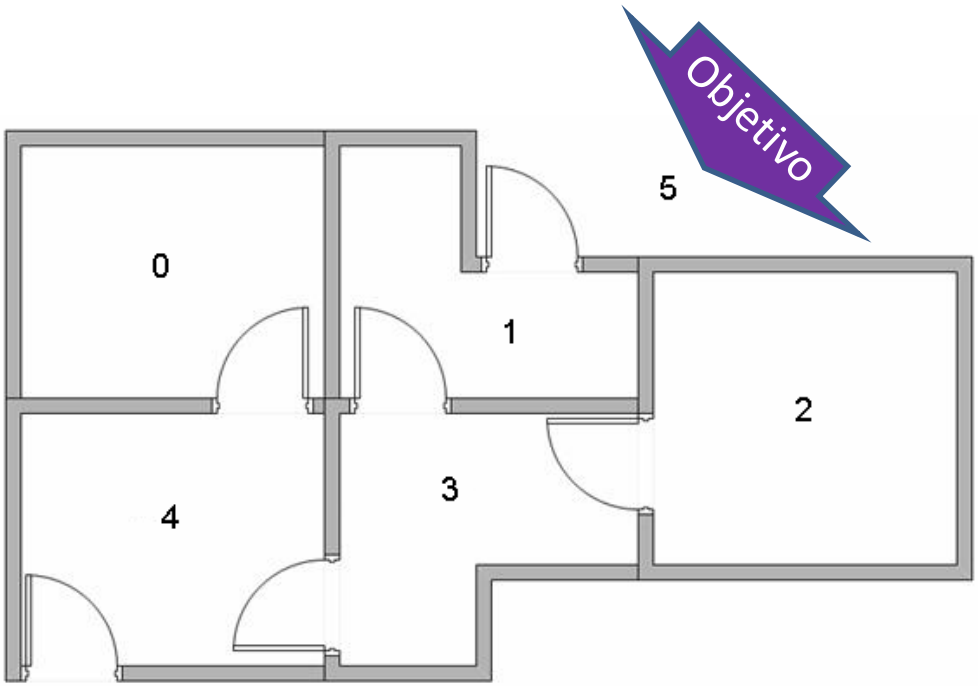
      8. $s \leftarrow s'$

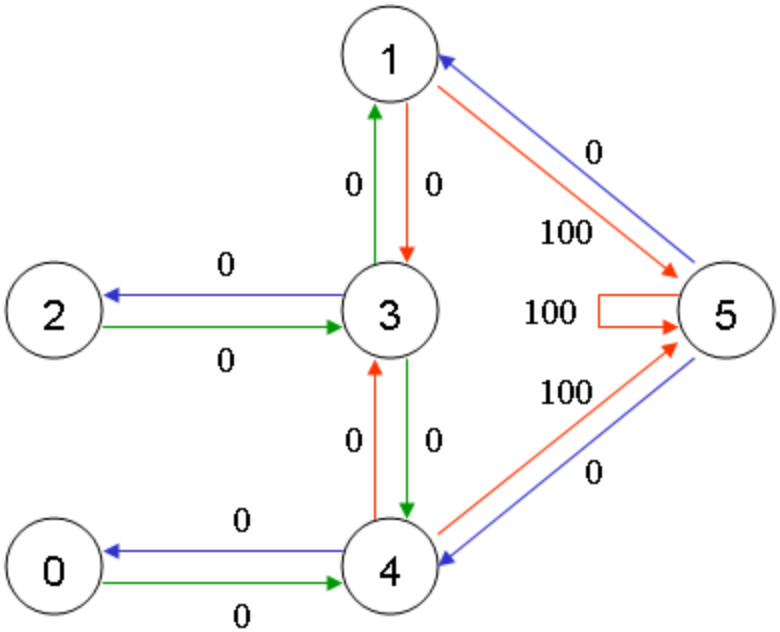
Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s, a] \longleftarrow (1-\alpha)Q[s, a] + \alpha(R(s,a,s') + \gamma \max_{a'} Q[s', a'])$$

$$s \longleftarrow s'$$



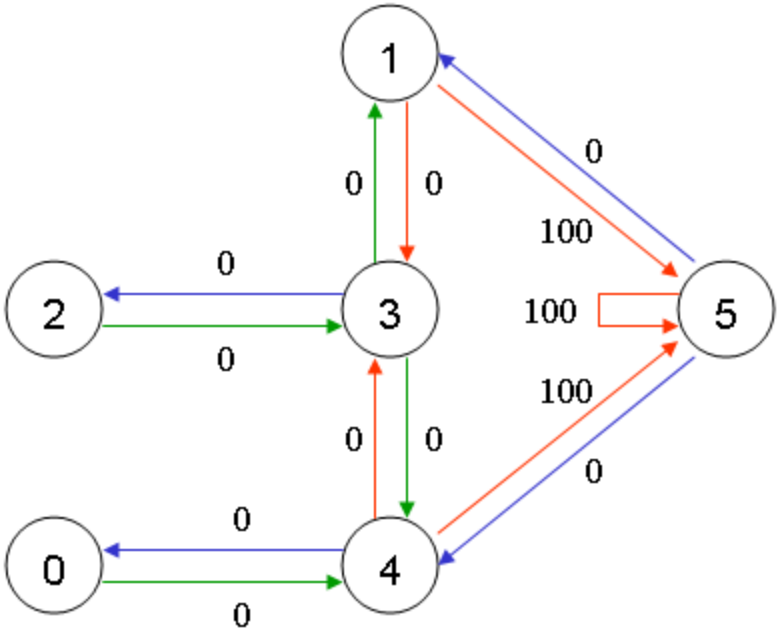


Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s,a] \longleftarrow (1\text{-}\alpha)Q[s,a] + \alpha(\, R(s,a,s') + \gamma\, max_{a'}\, Q[s',a'] \,)$$

$$s \longleftarrow s'$$



$$R = \begin{array}{c} \\ \text{Estado} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Ação} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \emptyset & \emptyset & \emptyset & & 0 & \emptyset \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & \emptyset \\ \emptyset & 0 & 0 & \emptyset & 0 & \emptyset \\ 0 & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & 0 & \emptyset & \emptyset & 0 & 100 \end{bmatrix} \end{array}$$

$$Q[s,a]$$

Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s,a] \longleftarrow (1\text{-}\alpha)Q[s,a] + \alpha(R(s,a,s') + \gamma \, max_{a'} \, Q[s',a'])$$

$$s \longleftarrow s'$$

$\gamma = 1$, $\alpha = 0.8$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{Estado} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Ação} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \emptyset & \emptyset & \emptyset & & 0 & \emptyset \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & \\ \emptyset & 0 & 0 & \emptyset & \emptyset & \emptyset \\ 0 & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & 0 & \emptyset & \emptyset & 0 & 100 \end{bmatrix} \end{array}$$

s = 1

Q[1,5] = 0.2 * Q[1,5] + 0.8 * (R[1,5,5] + 1 * max { Q[5,1], Q[5,4], Q[5,5] })

= 0 + 0.8 * (100 + 1 * 0) = 80

s = 5 ⟵ <u>Estado objetivo → encerra episódio!</u>

Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s,a] \longleftarrow (1\text{-}\alpha)Q[s,a] + \alpha(R(s,a,s') + \gamma \, max_{a'} \, Q[s',a'])$$

$$s \longleftarrow s'$$

$\gamma = 1$, $\alpha = 0.8$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 80 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{Estado} \\ \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Ação} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \emptyset & \emptyset & \emptyset & & 0 & \emptyset \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & \\ \emptyset & 0 & 0 & \emptyset & \emptyset & \emptyset \\ 0 & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & 0 & \emptyset & \emptyset & 0 & 100 \end{bmatrix} \end{array}$$

s = 3

Q[3,1] = 0.2 * Q[3,1] + 0.8 * (R[3,1,1] + 1 * max { Q[1,3], Q[1,5] })

= 0 + 0.8 * (0 + 1 * 80) = 64

s = 1

Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s,a] \longleftarrow (1\text{-}\alpha)Q[s,a] + \alpha(R(s,a,s') + \gamma\, max_{a'}\, Q[s',a'])$$

$$s \longleftarrow s'$$

$\gamma = 1,\ \alpha = 0.8$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 8 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{Estado} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Ação} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \emptyset & \emptyset & \emptyset & & 0 & \emptyset \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & \\ \emptyset & 0 & 0 & \emptyset & \emptyset & \emptyset \\ 0 & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & 0 & \emptyset & \emptyset & 0 & 100 \end{bmatrix} \end{array}$$

s = 1

Q[1,3] = 0.2 * Q[1,3] + 0.8 * (R[1,3,3] + 1 * max { Q[3,1], Q[3,2], Q[3,4] })

= 0.2 * 0 + 0.8 * (0 + 1 * 64) = 51.2

s = 3

Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s,a] \longleftarrow (1\text{-}\alpha)Q[s,a] + \alpha(R(s,a,s') + \gamma \, max_{a'} \, Q[s',a'])$$

$$s \longleftarrow s'$$

$\gamma = 1$, $\alpha = 0.8$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 8 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{Estado} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Ação} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \emptyset & \emptyset & \emptyset & & 0 & \emptyset \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & \\ \emptyset & 0 & 0 & \emptyset & \emptyset & \emptyset \\ 0 & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & 0 & \emptyset & \emptyset & 0 & 100 \end{bmatrix} \end{array}$$

s = 3

Q[3,2] = 0.2 * Q[3,2] + 0.8 * (R[3,2,2] + 1 * max { Q[2,3] })

= 0.2 * 0 + 0.8 * (0 + 1 * 0) = 0

s = 2

Q-Learning

execute ação *a*, observe recompensa *r* e novo estado *s'* (amostra)

$$Q[s,a] \longleftarrow (1\text{-}\alpha)Q[s,a] + \alpha\left(R(s,a,s') + \gamma \, max_{a'} \, Q[s',a'] \right)$$

$$s \longleftarrow s'$$

$\gamma = 1$, $\alpha = 0.8$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 8 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{Estado} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Ação} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} \emptyset & \emptyset & \emptyset & & 0 & \emptyset \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & \emptyset & \emptyset & 0 & \emptyset & \\ \emptyset & 0 & 0 & \emptyset & 0 & \emptyset \\ 0 & \emptyset & \emptyset & 0 & \emptyset & 100 \\ \emptyset & 0 & \emptyset & \emptyset & 0 & 100 \end{bmatrix} \end{array}$$

s = 2

Q[2,…] = ….

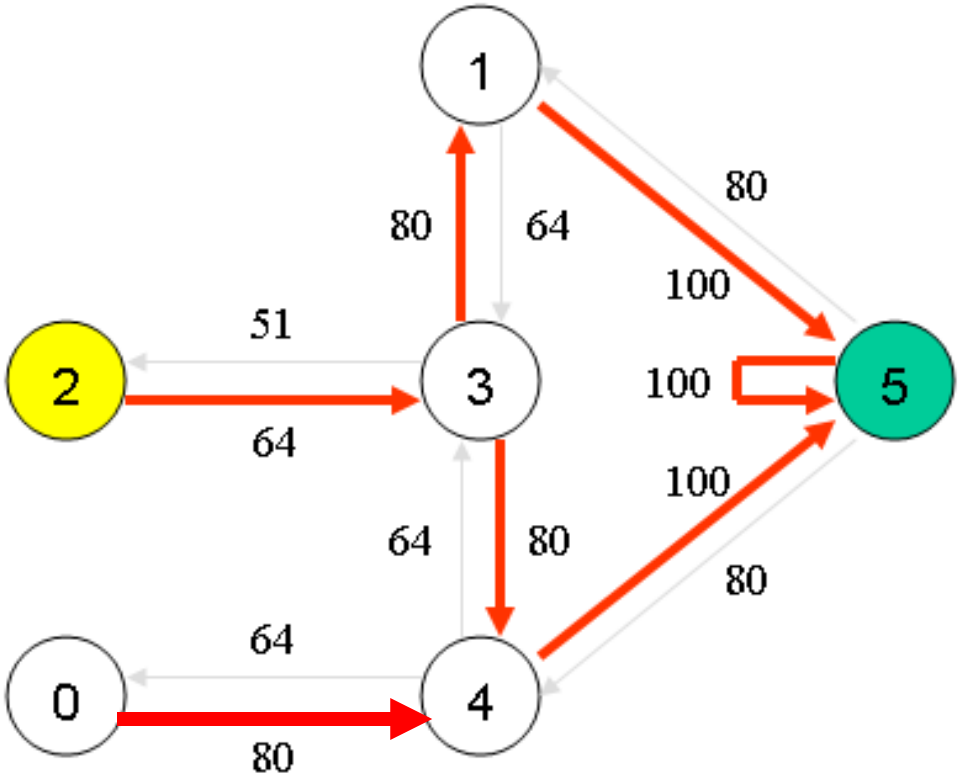
s = ….

Q-Learning

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{array}$$

$$Q[s, a]$$

Política aprendida



Exploração vs. Explotação

- O algoritmo de Q-learning envolve um equilíbrio entre Exploração (*Exploration*) e Explotação (*Explotation*).
- O agente precisa **explorar** o ambiente para descobrir novas ações e estados, o que pode levar à **descoberta de estratégias melhores**.
- Por outro lado, o agente também precisa **explotar** o conhecimento adquirido, utilizando as **ações que já sabe que geram boas recompensas**.
- A exploração excessiva pode levar a um aprendizado ineficiente, enquanto a explotação excessiva pode fazer com que o agente fique preso em estratégias subótimas.
- Técnica para resolver o dilema: **política** **ε-greedy**

Algoritmo Q-learning ε-greedy

1. Inicialize $Q(s, a)$ arbitrariamente para todos os estados s e ações a
2. Para cada episódio:
   3. Selecione um estado s aleatoriamente
   4. Enquanto s não for um estado terminal:
      5. **Escolha ação aleatória α com prob ε ou melhor ação α com prob $1 - \varepsilon$**
      6. Execute a ação α e observe a recompensa R e o próximo estado s' (amostra)
      7. Atualize $Q(s, a)$ usando a equação de Bellman:

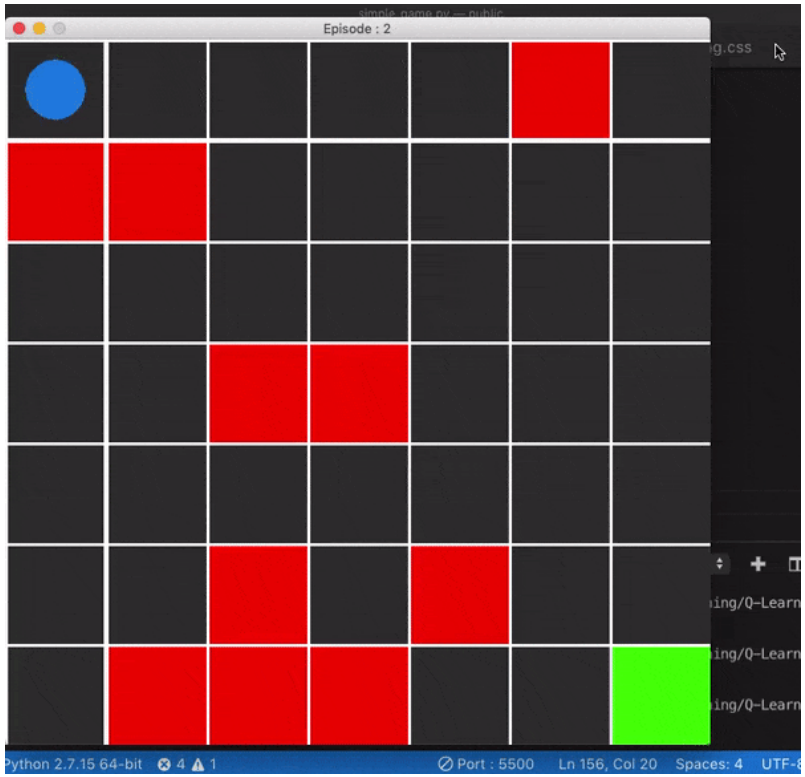
$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} Q(s', a')]$$

      8. $s \leftarrow s'$

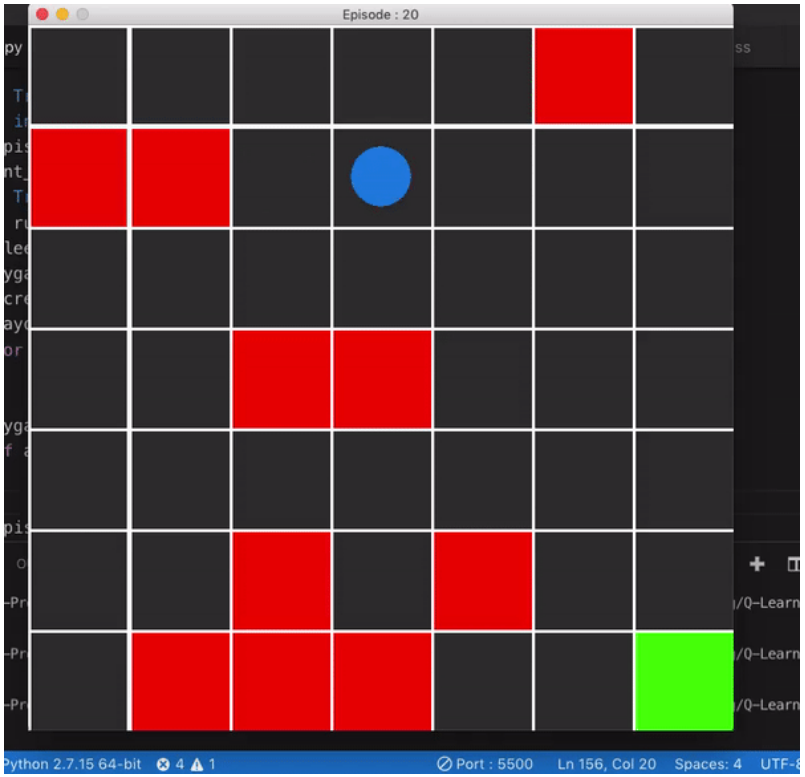
Exemplo de Q-learning: Labirinto

- Um exemplo clássico de aplicação de Q-learning é o problema do labirinto. O agente precisa navegar em um labirinto para encontrar a saída. O agente recebe uma **recompensa positiva ao encontrar a saída** e uma pequena **penalidade a cada passo** dado.

Iterações 2, 3, 4 e 5



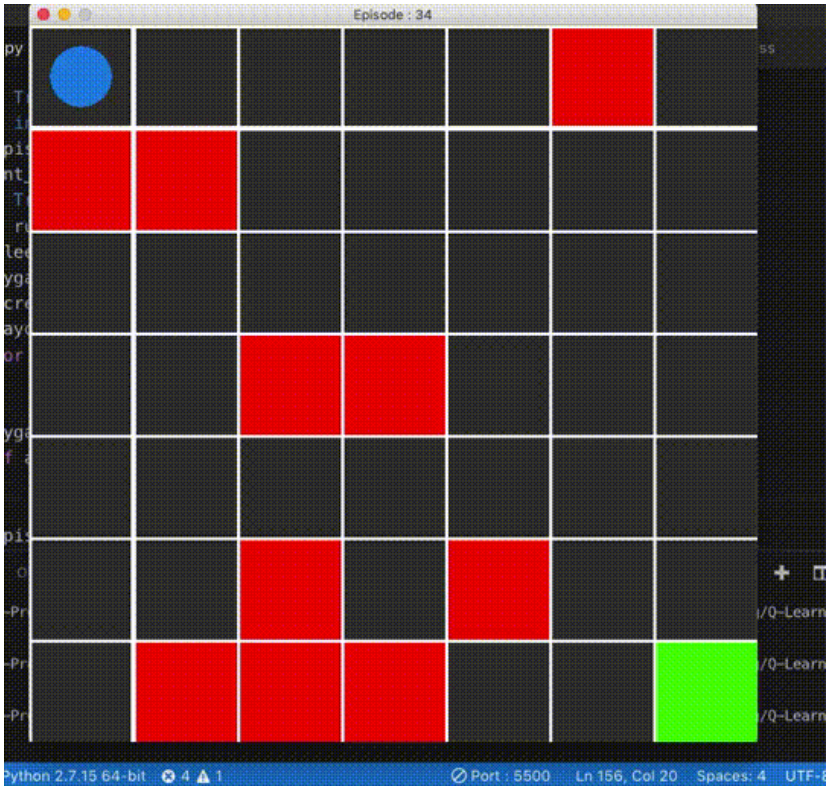
Iteração 20



Exemplo de Q-learning: Labirinto

- Um exemplo clássico de aplicação de Q-learning é o problema do labirinto. O agente precisa navegar em um labirinto para encontrar a saída. O agente recebe uma recompensa positiva ao encontrar a saída e uma pequena penalidade a cada passo dado.

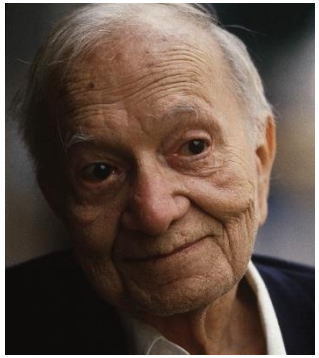
Iteração 36



Dúvidas?

"Livros não mudam o mundo, quem muda o mundo são as pessoas. Os livros só mudam as pessoas."

— Mario Quintana



- [AIMA] RUSSELL, Stuart J.; NORVIG, Peter. Artificial Intelligence: A Modern Approach. 4. ed. Global Edition. Harlow: Pearson, 2021.
  - Capítulos 16 (MDP) e 23 (AP. Reforço)
- [Website Hendrik] https://hendrikdcomp.github.io/ia/
- [Artigos Hendrik]
  - Sobre video-games e Aprendizado por Reforco: http://www.saense.com.br/2016/02/a-maquina-que-aprende-a-jogar-melhor-que-voce/
  - Sobre Minecraft e Aprendizado por Reforco: http://www.saense.com.br/2016/08/vida-longa-no-minecraft/
  - Sobre Poker e Equilíbrio de Nash: http://www.saense.com.br/2017/03/maquina-vence-campeoes-humanos-no-poker/